WEB TEXNOLOGIYALAR
LABORATORIYA ISHI ■ 15
Mavzu: WebSocket orqali oddiy real vaqt chat ilovasi yaratish

---

1. Ishning maqsadi

- WebSocket protokoli va uning HTTP dan farqini o'rganish
- Node.js va ws kutubxonasi yordamida WebSocket server yaratish
- Brauzer WebSocket API si bilan ishlash
- Real vaqt (real-time) ikki tomonlama aloqa mexanizmini tushunish
- Ko'p foydalanuvchili chat ilovasini amalda yaratish
- Ulanish holati, xabar yuborish va uzilishni boshqarish
- JSON orqali tuzilgan ma'lumot almashishni o'zlashtirish

2. Nazariy qism

WebSocket nima?

WebSocket — brauzer va server o'rtasida doimiy, ikki tomonlama aloqa kanalini ta'minlovchi protokol (RFC 6455). HTTP dan farqli ravishda WebSocket ulanish bir marta o'rnatiladi va uzilmaguncha ikki tomon ham istalgan vaqtda ma'lumot yubora oladi. Bu uni real vaqt ilovalar: chat, onlayn o'yinlar, moliyaviy kurschalar va jonli monitoring uchun ideal qiladi.

HTTP va WebSocket taqqoslash:

Xususiyat	HTTP (so'rov-javob)	WebSocket (doimiy kanal)
Ulanish turi	Har so'rovda yangi ulanish	Bir marta ulanib turadi
Ma'lumot oqimi	Faqat mijoz so'raganda	Ikki tomon ham yuboradi
Sarlavha (header)	Har so'rovda takrorlanadi	Faqat birinchi■handshake da
Kechikish	Yuqori (har so'rovda)	Minimal (doimiy kanal)
Real-time	Polling kerak (noqulay)	To'g'ridan-to'g'ri
Protokol	http:// yoki https://	ws:// yoki wss://
Qo'llanilishi	Oddiy veb-sahifalar	Chat, o'yin, monitor

WebSocket ulanish bosqichlari:

• Handshake — mijoz HTTP so'rov yuborib, WebSocket ga o'tishni so'raydi
• Upgrade — server roziligi bilan protokol ws:// ga o'tkaziladi
• Open — doimiy kanal ochilib, ikki tomon ma'lumot almasha boshlaydi
• Message — server yoki mijoz istalgan vaqtda xabar yuboradi
• Close — biror tomon ulanishni yopadi (close frame yuboriladi)

WebSocket hodisalari (brauzer API):

Hodisa / Metod	Qachon ishlaydi / Vazifasi
onopen	Server bilan ulanish muvaffaqiyatli o'rnatilganda

onmessage	Serverdan yangi xabar kelganda
onerror	Ulanishda xato yuz berganda
onclose	Ulanish yopilganda (server yoki mijoz tomonidan)
send(data)	Serverga ma'lumot yuborish metodi
close()	Ulanishni yopish metodi
readyState	Joriy holat: 0=CONNECTING, 1=OPEN, 2=CLOSING, 3=CLOSED

3. Loyiha tuzilishi

```
websocket-chat/
server/
server.js <- Node.js WebSocket server
package.json <- npm konfiguratsiya
client/
index.html <- Chat interfeysi
style.css <- Uslublar
chat.js <- Brauzer WebSocket mijozi
```

O'rnatish va ishga tushirish:

```
# Node.js o'rnatilganini tekshirish
node --version

# Loyiha papkasiga kirish
cd websocket-chat/server

# package.json yaratish
npm init -y

# ws kutubxonasini o'rnatish
npm install ws

# Serverni ishga tushirish
node server.js

# Brauzerda: index.html faylini oching
```

4. Amaliy qism — bosqichma-bosqich bajarish

1-bosqich: server.js — WebSocket server

ws kutubxonasi Node.js da WebSocket server yaratishning eng sodda yo'li. Har bir ulangan mijoz clients to'plamida saqlanadi.

```
const WebSocket = require('ws');

const PORT = 8080;
const server = new WebSocket.Server({ port: PORT });

// Ulangan foydalanuvchilar to'plami
```

```
const clients = new Map(); // ws -> { ism, id }
let idSanagich = 1;

console.log(`WebSocket server ${PORT}-portda ishlamoqda...`);

server.on('connection', (ws) => {
const foydalanuvchiId = idSanagich++;
clients.set(ws, { ism: null, id: foydalanuvchiId });
console.log(`Yangi ulanish: #${foydalanuvchiId}`);

// Xabar qabul qilish
ws.on('message', (xabar) => {
try {
const malumot = JSON.parse(xabar);
const mijoz = clients.get(ws);

if (malumot.tur === 'kirish') {
// Foydalanuvchi ismini saqlash
mijoz.ism = malumot.ism;
clients.set(ws, mijoz);

// Hammaga kirish xabarini yuborish
hammagaYuborish({
tur : 'tizim',
matn : `${malumot.ism} chatga qo'shildi`,
vaqt : new Date().toLocaleTimeString('uz'),
sonUlangan: clients.size
}, null);

} else if (malumot.tur === 'xabar') {
// Hammaga xabar yuborish
hammagaYuborish({
tur : 'xabar',
ism : mijoz.ism,
matn : malumot.matn,
vaqt : new Date().toLocaleTimeString('uz')
}, null);
}

} catch (xato) {
console.error('Xabar parse xatosi:', xato);
}
});

// Ulanish uzilganda
ws.on('close', () => {
const mijoz = clients.get(ws);
clients.delete(ws);
```

```
if (mijoz && mijoz.ism) {
hammagaYuborish({
tur : 'tizim',
matn : `${mijoz.ism} chatdan chiqdi`,
vaqt : new Date().toLocaleTimeString('uz'),
sonUlangan: clients.size
}, null);
}
console.log(`Ulanish uzildi: #${mijoz?.id}`);
});
// Xato holati
ws.on('error', (xato) => {
console.error('WebSocket xatosi:', xato);
});
});
// Barcha ulanganlaraga xabar yuborish
function hammagaYuborish(malumot, bundan_tashqari) {
const json = JSON.stringify(malumot);
clients.forEach((info, ws) => {
if (ws !== bundan_tashqari && ws.readyState === WebSocket.OPEN) {
ws.send(json);
}
});
// O'ziga ham yuborish (agar bundan_tashqari null bo'lsa)
if (!bundan_tashqari) {
clients.forEach((info, ws) => {
if (ws.readyState === WebSocket.OPEN) {
// Allaqachon yuborildi
}
});
}
}
```

2-bosqich: index.html — chat interfeysi

```
<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Real Vaqt Chat</title>
<link rel="stylesheet" href="style.css">
</head>
```

```
<body>

<!-- Kirish ekrani -->
<div id="kirishEkrani" class="ekran">
<div class="karta">
<h2>Chatga kirish</h2>
<input type="text" id="ismMaydoni"
placeholder="Ismingizni kiriting..." maxlength="20">
<button onclick="chatgaKirish()">Kirish</button>
</div>
</div>

<!-- Chat ekrani -->
<div id="chatEkrani" class="ekran yashirin">
<header>
<h2>Real Vaqt Chat</h2>
<div id="holatPanel">
<span id="ulanishHolati" class="holat">Ulanmoqda...</span>
<span id="ulananSoni">0 kishi</span>
</div>
</header>

<div id="xabarlarZona"></div>

<footer>
<input type="text" id="xabarMaydoni"
placeholder="Xabar yozing..." maxlength="200"
onkeypress="enterBosish(event)">
<button onclick="xabarYuborish()">Yuborish</button>
</footer>
</div>

<script src="chat.js"></script>
</body>
</html>
```

3-bosqich: style.css — chat dizayni

```
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: 'Segoe UI', sans-serif; background: #f0f2f5; height: 100vh; }

.ekran { display: flex; justify-content: center; align-items: center; height:
100vh; }
.yashirin { display: none !important; }

/* Kirish ekrani */
.karta {
background: white; padding: 32px; border-radius: 12px;
box-shadow: 0 4px 20px rgba(0,0,0,0.1); width: 320px; text-align: center;
```

```
}
.karta h2 { margin-bottom: 20px; color: #1a73e8; }

/* Chat ekrani */
#chatEkrani {
flex-direction: column; width: 100%; max-width: 700px;
height: 100vh; background: white;
}

header {
background: #1a73e8; color: white; padding: 14px 20px;
display: flex; justify-content: space-between; align-items: center;
}

#holatPanel { display: flex; gap: 12px; align-items: center; font-size: 13px; }

.holat { padding: 3px 10px; border-radius: 12px; font-weight: bold; }
.holat.ulanildi { background: #4CAF50; }
.holat.uzildi { background: #f44336; }

#xabarlarZona {
flex: 1; overflow-y: auto; padding: 16px;
display: flex; flex-direction: column; gap: 8px;
}

.xabar {
max-width: 70%; padding: 10px 14px;
border-radius: 18px; font-size: 14px; line-height: 1.4;
}
.xabar.men { align-self: flex-end; background: #1a73e8; color: white; }
.xabar.boshqa{ align-self: flex-start; background: #e9ecef; color: #333; }
.xabar.tizim {
align-self: center; background: transparent;
color: #888; font-size: 12px; font-style: italic;
}

.xabar .ism { font-weight: bold; font-size: 12px; margin-bottom: 3px; }
.xabar .vaqt { font-size: 11px; opacity: 0.7; margin-top: 3px; text-align: right;
}

footer {
padding: 12px 16px; border-top: 1px solid #eee;
display: flex; gap: 8px;
}

input {
flex: 1; padding: 10px 14px; border: 1px solid #ddd;
border-radius: 24px; font-size: 15px; outline: none;
}
```

```
input:focus { border-color: #1a73e8; }

button {
background: #1a73e8; color: white; border: none;
padding: 10px 20px; border-radius: 24px;
cursor: pointer; font-size: 15px;
}
button:hover { background: #1557b0; }
```

4-bosqich: chat.js — brauzer WebSocket mijozi

```
let ws;
let mening_ismim = '';

function chatgaKirish() {
const ismMaydoni = document.getElementById('ismMaydoni');
const ism = ismMaydoni.value.trim();

if (!ism) {
alert('Iltimos, ismingizni kiriting!');
return;
}

mening_ismim = ism;

// Ekranlarni almashtirish
document.getElementById('kirishEkrani').classList.add('yashirin');
document.getElementById('chatEkrani').classList.remove('yashirin');

// WebSocket ulanish o'rnatish
wsUlanish();
}

function wsUlanish() {
const holatEl = document.getElementById('ulanishHolati');
holatEl.textContent = 'Ulanmoqda...';
holatEl.className = 'holat';

ws = new WebSocket('ws://localhost:8080');

// Ulanish ochildi
ws.onopen = () => {
holatEl.textContent = 'Ulanildi';
holatEl.className = 'holat ulanildi';

// Serverni ismimiz haqida xabardor qilish
ws.send(JSON.stringify({ tur: 'kirish', ism: mening_ismim }));
document.getElementById('xabarMaydoni').focus();
};

// Xabar keldi
```

```
ws.onmessage = (event) => {
const malumot = JSON.parse(event.data);
xabarKorsat(malumot);
};

// Ulanish uzildi
ws.onclose = () => {
holatEl.textContent = 'Uzildi';
holatEl.className = 'holat uzildi';
tizimXabari('Server bilan aloqa uzildi. 5 soniyada urinib ko'riladi...');
setTimeout(wsUlanish, 5000); // Avtomatik qayta ulanish
};

// Xato
ws.onerror = (xato) => {
console.error('WebSocket xatosi:', xato);
};
}

function xabarYuborish() {
const maydon = document.getElementById('xabarMaydoni');
const matn = maydon.value.trim();

if (!matn) return;
if (!ws || ws.readyState !== WebSocket.OPEN) {
alert('Server bilan aloqa yo'q!');
return;
}

// Serverga yuborish
ws.send(JSON.stringify({ tur: 'xabar', matn: matn }));

// O'zimga ko'rsatish
xabarKorsat({
tur : 'xabar',
ism : mening_ismim,
matn: matn,
vaqt: new Date().toLocaleTimeString('uz'),
men : true
});

maydon.value = '';
maydon.focus();
}

function xabarKorsat(malumot) {
const zona = document.getElementById('xabarlarZona');
const div = document.createElement('div');
```

```
if (malumot.tur === 'tizim') {
div.className = 'xabar tizim';
div.textContent = malumot.matn;
// Ulangan foydalanuvchilar sonini yangilash
if (malumot.sonUlangan !== undefined) {
document.getElementById('ulananSoni').textContent =
malumot.sonUlangan + ' kishi';
}
} else if (malumot.tur === 'xabar') {
const menmi = malumot.men || malumot.ism === mening_ismim;
div.className = 'xabar ' + (menmi ? 'men' : 'boshqa');
div.innerHTML = `
${!menmi ? '<div class="ism">' + malumot.ism + '</div>' : ''}
<div>${malumot.matn}</div>
<div class="vaqt">${malumot.vaqt}</div>
`;
}
zona.appendChild(div);
zona.scrollTop = zona.scrollHeight; // Pastga aylantirish
}
function tizimXabari(matn) {
xabarKorsat({ tur: 'tizim', matn: matn });
}
function enterBosish(event) {
if (event.key === 'Enter') xabarYuborish();
}
```

5. JSON xabar protokoli

Xabar turi	Yuboruvchi	Maydonlar	Tavsifi
kirish	Mijoz → Server	{ tur, ism }	Foydalanuvchi ismini ro'yxatdan o'tkazish
xabar	Mijoz → Server	{ tur, matn }	Chat xabarini yuborish
xabar	Server → Mijozlar	{ tur, ism, matn, vaqt }	Xabarni hammaga tarqatish
tizim	Server → Mijozlar	{ tur, matn, vaqt, sonUlangan }	Kirish/chiqish tizim xabari

6. Real-time texnologiyalarni taqqoslash

Texnologiya	Ulanish	Yo'nalish	Kechikish	Qo'llanilishi
WebSocket	Doimiy	Ikki tomonlama	Minimal	Chat, o'yin, monitor
HTTP Polling	Har N soniya	Bir tomonlama	Yuqori	Oddiy yangilanish

SSE	Doimiy	Server → Mijoz	Past	Yangiliklar, log
Long Polling	Uzoq so'rov	Bir tomonlama	O'rta	Eski brauzerlar

7. Topshiriq

Mavzu: Xona asosidagi chat — foydalanuvchi xona tanlagan holda chat yarating.

Qo'shimcha xususiyatlar:

- Kirish ekranida ism + xona nomi (Umumiy, Texnologiya, Sport) tanlansin
- Har bir xona mustaqil bo'lib, faqat o'sha xona a'zolari xabarni ko'rsin
- Server tomonida xonalar Map> tarzida boshqarilsin
- Ulangan foydalanuvchilar soni xona bo'yicha ko'rsatilsin
- Foydalanuvchi xona o'zgartirganda eski xonaga 'chiqdi', yangisiga 'qo'shildi' xabari borsin

Bajarish bosqichlari:

1-bosqich: Node.js va ws kutubxonasini o'rnating

2-bosqich: server.js da xonalar tizimini Map yordamida yarating

3-bosqich: xonaYuborish() funksiyasini yozing — faqat xona a'zolariga yuborsin

4-bosqich: index.html da xona tanlash maydonini qo'shing

5-bosqich: chat.js da kirish xabariga xonaNomi maydonini qo'shing

6-bosqich: Har bir xona uchun ulangan sonni header da ko'rsating

7-bosqich: Ikki brauzer oynasida sinab ko'ring — xonalar mustaqil ishlashini tekshiring

8. Natija va Xulosa

Laboratoriya davomida quyidagilar bajarildi:

- WebSocket protokoli va HTTP dan farqlari o'rganildi
- Node.js va ws kutubxonasi yordamida WebSocket server yaratildi
- Brauzer WebSocket API (onopen, onmessage, onclose, send) o'zlashtirildi
- Ko'p foydalanuvchili real vaqt chat ilovasi qurildi
- JSON asosidagi xabar protokoli ishlab chiqildi
- Ulanish, uzilish va avtomatik qayta ulanish mexanizmi amalga oshirildi
- Chat interfeysi CSS bilan chiroyli ko'rinishga keltirildi

Xulosa: WebSocket protokoli real vaqt ilovalar yaratish uchun eng samarali texnologiyalardan biri hisoblanadi. HTTP dan farqli ravishda doimiy ochiq kanal orqali ishlashi kechikishni minimal darajaga tushiradi. Node.js va ws kutubxonasi yordamida server tomonida, brauzer WebSocket API si yordamida esa mijoz tomonida to'liq real vaqt aloqa tizimi qurildi. Bu texnologiya chat ilovalar, onlayn o'yinlar, jonli dashboard va boshqa ko'plab zamonaviy ilovalar uchun asos bo'lib xizmat qiladi.